

PROJET ECOTRACK

ARCHITECTURE LOGICIELLE

Mastère EADL — Filière Développement Logiciel (RNCP 38822)

I. VISION GLOBALE DE L'ARCHITECTURE

✓ Critères RNCP Ce1.3.1 / A2.1 : Architecture complète et cohérente

Figure 1 — Schéma d'architecture global d'ECOTRACK (voir Annexe A).

Le schéma global d'ECOTRACK se lit de gauche à droite en quatre plans. À gauche, le parc de 2 000 conteneurs connectés, répartis sur 12 secteurs d'une métropole de 500 000 habitants, publie une mesure de remplissage toutes les 15 minutes (soit environ 500 000 messages par jour) vers un broker MQTT Mosquitto, qui alimente le bus événementiel Apache Kafka. Au centre, une API Gateway assure la façade unique : terminaison TLS, authentification, rate limiting et routage vers une grappe de microservices conteneurisés, organisés en Bounded Contexts (Ingestion IoT, Optimisation des tournées, Facturation, Gamification, Signalement citoyen, Authentification). Ces services partagent une couche de persistance PostgreSQL 15 enrichie de PostGIS (géospatial) et un cache Redis. À droite, trois applications clientes — le tableau de bord web React des gestionnaires et administrateurs, l'application mobile Citoyen et l'application mobile Agent (React Native) — consomment les API via la Gateway et reçoivent les mises à jour temps réel. L'ensemble est containerisé avec Docker, orchestré par Docker Compose en développement et ciblant Kubernetes en M2.

ECOTRACK repose sur une architecture microservices structurée par le Domain-Driven Design. Plutôt qu'un monolithe, le système est découpé en Bounded Contexts qui correspondent aux domaines métier réels du projet : l'Ingestion IoT (réception et validation des mesures capteurs), l'Optimisation des tournées (calcul d'itinéraires TSP/2-opt), la Facturation (suivi des collectes et coûts), la Gamification (points, badges, défis, classements — module central côté citoyen), le Signalement citoyen (remontées terrain géolocalisées) et l'Authentification (gestion des comptes et des rôles). Ce découpage répond aux trois contraintes structurantes du projet : un fort volume de données temps réel (ingestion MQTT/Kafka continue), des traitements hétérogènes (CRUD métier, calcul d'optimisation intensif, requêtes géospatiales) et un besoin de scalabilité indépendante par domaine. On peut ainsi monter en charge le seul service réellement sollicité —

typiquement l'Ingestion IoT ou l'Optimisation — sans surdimensionner l'ensemble.

En façade, une API Gateway joue le rôle de point d'entrée unique : elle centralise la terminaison TLS, la validation des jetons JWT, le contrôle RBAC, le rate limiting (appuyé sur Redis) et le routage vers le bon microservice. Derrière la Gateway, la communication inter-services est majoritairement asynchrone et événementielle, portée par le bus Apache Kafka : les producteurs publient des événements métier (mesure reçue, seuil franchi, signalement créé, tournée planifiée, points attribués) sur des topics dédiés, et chaque service consomme les topics qui le concernent. Ce modèle découple fortement les services, absorbe les pics de charge IoT et permet de rejouer les flux. La persistance s'appuie sur PostgreSQL 15 + PostGIS (données métier et géospatiales, index GiST) et sur Redis (cache, sessions, compteurs de rate limiting). Tous les composants sont containerisés sous Docker ; l'orchestration Kubernetes constitue la cible de la phase M2.

Les interactions suivent un flux clair. Les capteurs des 2 000 conteneurs émettent une mesure toutes les 15 minutes vers le broker MQTT, qui la dépose sur le topic Kafka d'ingestion ; le service Ingestion IoT consomme ce flux, valide et persiste les mesures, puis publie un événement d'alerte lorsqu'un seuil de remplissage est franchi. Le tableau de bord des gestionnaires reçoit ces événements en temps réel et met à jour la cartographie. Lorsqu'un gestionnaire planifie une collecte, le service Optimisation calcule l'itinéraire le plus court (algorithme TSP / 2-opt), persiste la tournée et publie un événement consommé par l'application mobile Agent. Le service Signalement citoyen reçoit les remontées de l'application mobile Citoyen, crédite des points via le service Gamification et notifie le gestionnaire de zone.

L'Authentification, partagée par toutes les applications, gère les quatre rôles RBAC, garantissant une séparation nette des responsabilités.

A. Principes Architecturaux

- Modularité (DDD) : un microservice par Bounded Context, chacun maître de son schéma de données et déployable indépendamment, ce qui isole les évolutions et les pannes par domaine.

- Scalabilité horizontale : chaque service est sans état et répliquable derrière l'API Gateway ; le bus Kafka et le cache Redis absorbent les pics, l'Ingestion IoT et l'Optimisation montant en charge indépendamment du reste.
- Résilience : communication asynchrone par événements (Kafka), pattern Circuit Breaker sur les appels synchrones inter-services et dégradation gracieuse (réponses en cache, files de rattrapage) pour viser un uptime de 99,5 %.
- Sécurité applicative : façade unique via l'API Gateway (TLS, JWT, RBAC 4 rôles, rate limiting), moindre privilège entre services et journalisation des accès (la sécurité réseau/infra relève des filières Cyber et Data).
- Maintenabilité : contrats d'API versionnés et documentés (OpenAPI/Swagger), tests automatisés (unitaires, intégration, contrat, e2e) et conteneurs Docker reproductibles d'un environnement à l'autre.

B. Vue d'Ensemble des Composants

- Couche Présentation : 3 applications clientes — Dashboard web React (gestionnaires/admins), App mobile Citoyen et App mobile Agent (React Native).
- Couche Métier : API Gateway en façade + microservices DDD (Ingestion IoT, Optimisation, Facturation, Gamification, Signalement, Authentification) communiquant via Kafka.
- Couche Données : PostgreSQL 15 + PostGIS (métier et géospatial, index GiST), Redis (cache/sessions/rate limiting), partitionnement temporel des mesures.
- Couche Infrastructure : containerisation Docker, bus événementiel Kafka, orchestration Kubernetes en cible M2 (le réseau et la supervision infra relèvent des filières Cyber et Data).

II. ARCHITECTURE DÉTAILLÉE PAR FILIÈRE

Architecture Microservices orientée Domain-Driven Design (Filière Développement)

A. Couche Présentation — 3 applications clientes

1. Dashboard web (gestionnaires et administrateurs)

- Framework : React 18 avec TypeScript
- State Management : Redux Toolkit + RTK Query
- Routing : React Router v6
- UI Library : Material-UI (MUI) + Tailwind CSS
- Cartographie : Leaflet + React-Leaflet (fonds OpenStreetMap)
- Temps réel : Socket.io-client pour WebSocket
- Charts : Recharts pour visualisations

2. Application mobile Citoyen (React Native)

- Framework : React Native
- Fonctionnalités : Scan QR codes, Géolocalisation, Mode offline

B. Couche Métier — API Gateway + microservices DDD

L'API Gateway constitue la façade unique du système : terminaison TLS, vérification du JWT, contrôle RBAC, rate limiting (Redis) et routage vers le bon microservice. Derrière elle, six microservices correspondant aux Bounded Contexts du domaine communiquent de façon asynchrone via le bus Apache Kafka (producteurs/consommateurs sur topics dédiés) et, lorsque nécessaire, par appels HTTP/gRPC internes protégés par Circuit Breaker :

1. Service Ingestion IoT (Bounded Context Ingestion IoT, Node.js)

- Ingestion MQTT : 2 000 capteurs, 1 mesure toutes les 15 min (~500 000 messages/jour), MQTT Mosquitto → topic Kafka
- Validation données : Schéma JSON
- Déclenchement alertes : Seuils configurables

2. Service Cœur métier / API REST (Python FastAPI)

- Endpoints CRUD : Conteneurs, Tournées, Alertes, Utilisateurs
- Documentation : OpenAPI/Swagger
- Validation : Pydantic models
- Rate limiting : 1000 req/min par user

3. Service Optimisation des tournées (Bounded Context Optimisation, Python)

- Algorithme TSP : Génétique OU glouton

- Contraintes : capacité véhicule, horaires, zones
 - Performance : <30s pour tournée 50 conteneurs
4. Service Authentification (Bounded Context Authentication, Node.js)
- JWT tokens : Access (15 min) + Refresh (7 jours)
 - 2FA : TOTP (Google Authenticator)
- RBAC : 4 rôles (Citoyen, Agent, Gestionnaire, Administrateur), principe de moindre privilège
5. Service Gamification + Signalement (Bounded Contexts Gamification et Signalement citoyen, Node.js)
- Email : Nodemailer + SendGrid
 - Signalement citoyen : remontées géolocalisées (déduplication <1 h), crédit de points, notification au gestionnaire de zone
 - Push notifications : Firebase Cloud Messaging
- C. Couche Données — persistance polyglotte légère
1. PostgreSQL 15 + PostGIS (base principale)
- Tables : users, containers, measurements, routes, alerts
 - Index géospatiaux : GiST pour requêtes spatiales rapides
 - Partitionnement : measurements par mois
2. Redis (cache et sessions)
- Cache API responses : TTL 5-60 min
 - Sessions utilisateurs
 - Rate limiting counters
3. Apache Kafka (bus événementiel)
- Séries temporelles mesures IoT
 - Agrégations automatiques (1h, 1j, 1 semaine)
 - Compression données anciennes
- D. Patterns de Conception Utilisés
- Repository : chaque microservice isole son accès aux données derrière une interface de dépôt (ex. ConteneurRepository, MeasureRepository). La logique métier ignore le détail SQL/PostGIS, ce qui facilite les tests (dépôts simulés) et l'évolution du schéma sans impacter les couches supérieures.
 - Strategy : le service Optimisation expose une interface StrategieOptimisation implémentée par plusieurs algorithmes TSP (gloutonne pour le MVP, 2-opt et génétique en évolution). La stratégie est sélectionnée à l'exécution selon la taille de la tournée et le budget

temps, sans modifier le code appelant — clé pour faire évoluer la qualité d'optimisation sans régression.

- Observer / event-driven (Kafka) : les services publient des événements métier sur des topics ; les consommateurs intéressés y réagissent de façon découplée. Côté clients, le dashboard s'abonne aux flux temps réel (remplissage, alertes) poussés via WebSocket, sans polling, maintenant la carte à jour.
- API Gateway : point d'entrée unique en façade des microservices ; centralise TLS, authentification JWT, contrôle RBAC, rate limiting et routage, et masque la topologie interne aux applications clientes.
- Circuit Breaker (résilience, ex. Resilience4j) sur les appels synchrones inter-services pour éviter les pannes en cascade ; CQRS possible côté lecture analytics (séparation des modèles écriture/lecture pour les KPIs gestionnaire). Les préoccupations transverses (auth, journalisation, gestion d'erreurs) sont traitées en middlewares chaînés à l'API Gateway et dans chaque service.

III. SCHÉMAS D'ARCHITECTURE

Schémas Obligatoires :

1. Architecture Globale : Vue d'ensemble de tous les composants
2. Flux de Données : Parcours données de l'ingestion à l'affichage
3. Architecture de déploiement : conteneurs Docker, API Gateway et cible Kubernetes (M2)
4. Architecture Déploiement : Environnements (dev, staging, prod)
5. Schéma Base de Données : ERD avec relations
6. Diagramme de flux événementiel : topics Kafka, producteurs et consommateurs

Figure 1 — Architecture globale : vue d'ensemble des composants (capteurs MQTT, bus Kafka, API Gateway, microservices DDD, données, 3 applications clientes) — voir Annexe A.

Figure 2 — Flux de données temps réel : parcours d'une mesure du capteur jusqu'à l'affichage et l'alerte (capteur → MQTT → Kafka → service Ingestion IoT → PostgreSQL → WebSocket → dashboard) — voir Annexe B.

Figure 3 — Architecture de déploiement : conteneurs Docker, API Gateway, environnements dev / staging / production et cible d'orchestration Kubernetes (M2) — voir Annexe C.

Figure 4 — Modèle de données (ERD) : entités Utilisateur, Conteneur, Zone, Mesure, Tournée, Signalement, Alerte et Gamification (Point, Badge, Défi, Classement), avec index GiST PostGIS et partitionnement temporel des mesures — voir Annexe D.

Figure 5 — Diagramme de flux événementiel : topics Kafka (iot.measurements, alerts.raised, signals.created, routes.planned, gamification.points) et leurs producteurs/consommateurs — voir Annexe E.

Ces schémas, réalisés sous draw.io et PlantUML (C4 niveau 2), sont fournis en annexe au format vectoriel pour préserver leur lisibilité à l'impression.

IV. SCALABILITÉ ET PERFORMANCE

A. Stratégies de Scalabilité

1. Scalabilité Horizontale

- Load balancers : Distribution trafic sur N instances
- Microservices : scaling indépendant par Bounded Context derrière l'API Gateway
- BDD : réplicas en lecture PostgreSQL pour l'analytics ; cache Redis (hit >80 %)
- Bus Kafka : partitionnement des topics pour absorber les ~500 000 messages IoT/jour

2. Scalabilité Verticale

- Ressources : CPU, RAM, IOPS ajustables
- Cas d'usage : BDD principale, Spark master
- Limites : Plafond matériel (→ passer horizontal)

B. Objectifs de Performance

- Temps de réponse API : < 500 ms au 95e centile (p95)
- Requêtes SQL : < 200 ms pour 90 % des requêtes
- Charge : 500 utilisateurs simultanés supportés
- Cache : taux de hit Redis > 80 % sur les lectures fréquentes
- Disponibilité : uptime 99,5 % ($\approx$ 43 h d'indisponibilité/an au maximum)

C. Tests de Charge Prévus

- Outils : Apache JMeter, k6, Locust
- Scénarios : montée jusqu'à 500 utilisateurs simultanés (cible NFR), avec marge
- Fréquence : Sprint 3, 6, 8 (+ avant prod)
- Métriques : Throughput, latency, error rate

V. SÉCURITÉ ET CONFORMITÉ

✓ Critères RNCP Ce1.3.2 / A2.1 : Sécurité intégrée

A. Authentification et Autorisation

- Authentification : JWT (Access 15 min + Refresh 7j)
- MFA : TOTP obligatoire pour gestionnaires et administrateurs ; blocage après 5 tentatives
- RBAC : 4 rôles (Citoyen, Agent, Gestionnaire, Administrateur), principe de moindre privilège
- Sessions : Redis avec TTL
- Password : bcrypt (cost 12), min 12 caractères

B. Chiffrement

- Transport : TLS 1.3 obligatoire (HTTPS)
- At-rest : AES-256 pour données sensibles (BDD)
- Secrets : Vault (HashiCorp) ou AWS Secrets Manager
- Certificats : Let's Encrypt (auto-renew)

C. Conformité RGPD

- Consentement : Opt-in explicite utilisateurs
- Droit accès : API GET /me/data
- Droit effacement : Soft delete + purge 90j
- Portabilité : Export JSON/CSV
- Minimisation : Données strictement nécessaires
- DPO : Désigné + registre traitements

D. Recommandations ANSSI

- Guide Sécurité Cloud : Chiffrement, cloisonnement
- Bonnes pratiques DevSecOps : Scan vulnérabilités
- Journalisation : Logs 1 an, audit trail

FIN DE L'ARCHITECTURE LOGICIELLE
